



bKash presents SUST CSE Carnival 2026

Codex Community Hackathon

In association with Codex and Poridhi.io

Online Preliminary Round

Team Instructions Manual

Table of Contents

Team Instructions Manual-----	2
1. Participant Document Pack-----	3
2. What Teams Need to Build-----	3
3. Available Resources-----	3
4. Suggested Team Role Split-----	4
5. API Submission Rule-----	4
6. Deployment Options-----	4
7. Deploying on Poridhi Lab / VM / AWS-----	4
8. Docker Fallback Rules-----	5
9. AI and Model Usage Policy-----	5
10. Secrets and Environment Variables-----	6
11. Repository Access Policy-----	7
12. Testing Checklist Before Submission-----	7
13. Submission Form Checklist-----	7
14. What Not to Do-----	8
15. Common Troubleshooting-----	8
16. Final Pre-Submit Checklist-----	9

Team Instructions Manual

Read this first

This manual explains how to execute the preliminary round: read the problem, divide work, build the API, test it, deploy it, and submit the required deliverables. It should be read together with the Problem Statement and the Evaluation Rubric.

1. Participant Document Pack

Document	Purpose	What it answers
Problem Statement	Defines the challenge, input/output schema, and required behavior.	What do we need to build?
Evaluation Rubric	Explains scoring categories, safety penalties, hidden tests, and tie-breakers.	How will we be judged?
Team Instructions Manual	Explains build flow, deployment options, secrets policy, testing, and submission.	How do we execute and submit?

2. What Teams Need to Build

Required item	Instruction
API service	Build a backend service for QueueStorm Investigator.
GET /health	Must return {"status":"ok"}. This proves the service is running.
POST /analyze-ticket	Main endpoint. It must accept the problem statement input JSON and return the required structured output JSON.
Valid JSON response	Use the exact required field names, types, and enum values from the problem statement.
README.md	Explain setup, run command, AI/model usage, safety logic, and known limitations.

Frontend/UI is optional

A frontend or UI is not required for the preliminary round and will not be directly judged. Prioritize API correctness, evidence reasoning, safety, reliability, deployment, and documentation.

3. Available Resources

Resource	How teams may use it
Poridhi Labs	Use the provided lab environment for coding, testing, and deployment support.
Poridhi VM	Deploy the API service manually on a VM if provided.

Resource	How teams may use it
AWS through Poridhi Labs	Deploy using AWS resources available through Poridhi Labs, such as EC2 or similar environments.
Puku Editor/CLI	Use for AI-assisted coding, debugging, project setup, refactoring, and documentation.
Any other platform	Teams may also deploy on Render, Railway, Fly.io, Vercel, AWS EC2, or any other reachable hosting platform.

Resource policy

Poridhi resources are provided as support, not as a restriction. Teams may deploy anywhere they want as long as the submitted API is reachable and judgeable.

4. Suggested Team Role Split

Role	Main responsibility
API/Backend Lead	Build endpoints, request parsing, response formatting, validation, and deployment setup.
Reasoning/Logic Lead	Implement transaction matching, evidence verdict, case classification, routing, and severity.
AI/Safety/Docs Lead	Integrate LLM/rules/local model if used, add safety guardrails, test edge cases, and write README.

For solo teams: follow the same order - schema first, reasoning second, safety third, deployment last.

5. API Submission Rule

The judge should be able to call the following endpoints from the submitted base URL:

```
GET https://your-service-url.com/health
POST https://your-service-url.com/analyze-ticket
```

- No login, dashboard access, manual approval, or private network access should be required for the judge.
- The service must accept JSON input and return JSON output.
- Use the exact endpoint names from the problem statement.
- The service should remain reachable during the evaluation window.

6. Deployment Options

Priority	Submission path	What to submit	Notes
1	Working endpoint URL	Public base URL and GitHub repository.	Preferred path. Judges call the API directly.

Priority	Submission path	What to submit	Notes
2	Lightweight Docker fallback	Dockerfile or image details, dependency files, and run command.	Accepted if public deployment is not possible.
3	Code-only reproducibility	GitHub repo with complete setup/run documentation.	Last fallback. May receive reduced deployment/reproducibility credit if hard to run.

7. Deploying on Poridhi Lab / VM / AWS

- Create the project repository and confirm that the API runs locally first.
- Use Poridhi Lab, Poridhi VM, or AWS through Poridhi Labs if provided to your team.
- Install dependencies on the VM or selected environment.
- Set required environment variables in the runtime environment, not in the repository.
- Run the service on the documented port and bind it to 0.0.0.0.
- Expose the service using the platform URL, VM public IP, reverse proxy, or any provided deployment mechanism. (Poridhi Labs Documentation is also provided to the teams)
- Test /health and /analyze-ticket from outside the environment before submitting.

8. Docker Fallback Rules

Rule	Requirement
Recommended image size	Under 500MB.
Hard image size limit	1GB.
GPU	Not allowed.
Large local model weights	Not allowed.
Multi-GB downloads during evaluation	Not allowed.
Runtime training	Not allowed.
Port binding	Must bind to 0.0.0.0.
Health readiness	/health must respond within 60 seconds of service start.
Secrets	Must be passed through environment variables only. Do not bake secrets into the image.

```
docker build -t queuestorm-team .
docker run -p 8000:8000 --env-file judging.env queuestorm-team
```

9. AI and Model Usage Policy

Allowed approach	Status
Rule-based logic	Allowed and encouraged. The task is designed to be solvable without paid APIs.
External AI APIs	Allowed using the team's own account and keys. Organizers will not provide third-party API keys.
Lightweight local models	Allowed if they run without GPU and fit within runtime/image limits.
Hybrid rule + AI system	Recommended. Use rules for evidence/safety and AI for language understanding or drafting.
Huge local LLMs / GPU dependency	Not allowed for preliminary judging.

Third-party API responsibility

If a team uses OpenAI, Anthropic, Hugging Face, Google AI, or any other external API, the team is responsible for API keys, cost, quota, rate limits, and availability during evaluation.

10. Secrets and Environment Variables

Important security rule

Do not commit real secrets to GitHub, even if the repository is private. Do not put secrets in README, screenshots, Docker images, commit history, or public messages.

Where	What should be placed there
GitHub repository	Source code, README, dependency files, Dockerfile if needed, and .env.example only. No real secrets.
.env.example	Variable names only. Example values should be placeholders.
Hosting platform	Real secrets for deployed endpoint submissions. Example: Render/Railway/Fly/Vercel/EC2/Poridhi Lab environment variables.
Submission form private field	Real secrets only if Docker/code fallback requires them for judging. This field should be visible only to technical judges.

Repository example:

```
OPENAI_API_KEY=  
MODEL_NAME=  
PORT=8000
```

Private judging secret example, only if required for Docker/code fallback:

```
OPENAI_API_KEY=your_real_temporary_key
MODEL_NAME=your_model_name
PORT=8000
```

- Teams should use temporary, limited-quota keys when sharing secrets for judging.
- Teams should revoke or rotate shared keys after evaluation is complete.
- Organizers will not provide third-party API keys for this round.
- If a Docker/code fallback depends on private secrets that are not provided, judges may not be able to run it fully and the team may lose deployment/reproducibility or functionality points.

11. Repository Access Policy

Repository type	Requirement
Public repository	Submit the repository URL in the form.
Private repository	Add the organizer GitHub handle(s) before the deadline with read access.
Repository availability	The repository must remain accessible to organizers until preliminary results are published.
After results	Teams may delete, archive, or make the repository private after preliminary results are published.
Secrets	The repository must not contain real secrets at any time.

12. Testing Checklist Before Submission

Check	Required?
/health returns {"status":"ok"}	Yes
/analyze-ticket accepts sample JSON	Yes
Response contains all required fields	Yes
Enum values match the problem statement exactly	Yes
Service handles empty or missing transaction history safely	Yes
Service handles malformed/non-critical missing fields without crashing	Yes
Customer reply does not ask for PIN, OTP, password, or secret credentials	Yes
Customer reply does not promise refund, reversal, recovery, or account unblock without authority	Yes
Endpoint or Docker fallback responds within timeout	Yes
README is complete	Yes

13. Submission Form Checklist

Field	Required?	Notes
Team name and team ID	Yes	Use the registered team information.
GitHub repository URL	Yes	Public or private, with organizer access.
Submission path	Yes	Endpoint / Docker fallback / Code-only reproducibility.
Public endpoint base URL	If the endpoint path	Example: https://team-app.example.com
Docker build/run command	If Docker fallback	Include expected port and env-file usage.
Required environment variable names	If applicable	Names only, not secret values.
Secrets for judging	Only if needed	Use the private form field, not GitHub.
Sample request and sample response	Yes	Can be in README or separate files.
AI/model usage explanation	Yes	Mention rules, local model, external API, or hybrid approach.
Safety logic explanation	Yes	Explain OTP/PIN/refund/reversal safeguards.
Known limitations	Yes	Be honest about edge cases and failure modes.
No real customer data confirmation	Yes	Only synthetic data should be used.
No secrets committed confirmation	Yes	Checkbox or written confirmation.

14. What Not to Do

Do not	Why
Do not build only a UI or screenshots	The preliminary round judges the API.
Do not submit an endpoint that requires login	The judge harness must call it directly.
Do not use real customer or payment data	Privacy and safety issue. Use only synthetic data.
Do not integrate real payment APIs	Out of scope for the preliminary round.
Do not ask users for OTP, PIN, password, or secret credentials	Critical fintech safety violation.
Do not promise refunds, reversals, account unblocks, or recovery	The system is a support copilot, not an authority.

Do not	Why
Do not commit API keys or .env files	Security risk and bad engineering practice.
Do not rely on huge models, GPU, or multi-GB downloads	Not judgeable at scale.

15. Common Troubleshooting

Problem	What to check
404 on /health or /analyze-ticket	Confirm exact route names and base URL.
Invalid JSON response	Return application/json and avoid printing extra logs in the response body.
Schema error	Check required fields, data types, enum spelling, and null handling.
Timeout	Reduce model calls, add fallback logic, cache where safe, and avoid large downloads.
External API failure	Handle quota/rate-limit errors safely and return a controlled response.
Docker runs locally but not for judges	Bind to 0.0.0.0, expose the correct port, and document the run command.
Private repo inaccessible	Add organizer GitHub handle(s) before the deadline.
Missing secrets	Use hosting env vars for deployed endpoint or private submission field for Docker/code fallback.

16. Final Pre-Submit Checklist

- Problem statement read and implementation aligned with the required schema.
- GET /health and POST /analyze-ticket tested successfully.
- Safety guardrails tested against OTP/PIN/refund/reversal cases.
- Endpoint deployed or Docker/code fallback prepared.
- GitHub repository accessible to organizers.
- README includes setup, run command, sample request, sample response, AI/model usage, safety logic, and limitations.
- .env.example added if environment variables are needed.
- No real secrets committed to the repository.
- Required private secrets submitted only through the official private field if needed for judging.
- Submission form completed before the deadline.

Final advice

Build the API first. Make the schema correct. Add evidence and reasoning. Add safety guardrails. Test it. Deploy it. Submit clearly. A simple, reliable, safe API will score better than a flashy but broken product.